A.消消乐

两种操作：

- 找一个 A，匹配之后的一个 B 并把它们删除。
- 找一个 B，匹配之后的一个 C 并把它们删除。

换一个角度想，在序列里找一个 B，与它前面的 A 或后面的 C 匹配并删除。

从左往右找，每找到一个 B，匹配之后的距离它最近的一个 C，若没有则匹配一个前面的 A。

由于每一个点只能被删一次，能匹配就匹配肯定是最优的。可以发现，能匹配的 A 是越来越多的，而 C 是越来越少的，所以优先匹配 C。又因为距离越近的 C 失效得越快，所以优先匹配距离近的 C。

B.修路

考虑只有 0 长度的边怎么做。

发现相当于我们要选一些边，代价最小且满足图连通，既最小生成树。

那么考虑剩下的边，并对他们按长度为第一关键字，代价为第二关键字，从小到大贪心，如果当前 u, v 最短路小于等于 l，那么这条边一定没用，否则一定有用。

而又基于长度大的边加入后不会对长度小的边（u, v 最短路小于等于 l）这个判断条件有影响，所以这样贪心是对的。

瓶颈在 m 次最短路，即复杂度 $O(m^2 \log m)$。

C.子序列

首先可以 dp，然后注意到可以用矩阵来表示，为了方便需要把 dp 状态设计为处理完前 i 个数，最后一个数是 x 的方案。

那么问题就变成了求区间矩阵乘积之后求矩阵一行的值的和。

可以做分治，合并答案在预处理之后是 $O(k^2)$ 的，预处理矩阵乘向量也是 k^2，所以总复杂度 $O(k^2(n \log n + q))$。

D.注意到每个时刻都必须做菜，我们可以设计一个 $f_{i,j}$ 表示分别做了 i, j 个 A, B 的最大答案。

那么可以得到 $sa_i + sb_j <= S_i$ 就可以得分，对应一个 $j <= pos_i$ 让 $i - 1$ 加一。

考虑对行（也就是 i）扫描，转移到下一行是一个前缀加，每一行内部的转移比较难，因为每一个列要加的行是一个前缀。

那么我们可以先考虑为 $f_{i,j-1} + s_{i,j} \to f_{i,j}$，对 s_i 作前缀和并减到 f_i 上，那么转移就是做前缀 checkmax。

那我们维护一个区间 checkmax 标记，每次 pushdown 的时候把左边的 max 贡献到右边，剩下的是一些区间加。